

SNAP UP

Pilot Implementation Task Specification

Customer Web App • Store Database • Session Renewal • Guest Checkout • Payment • Staff Verification

Purpose: This document consolidates the latest pilot requirements into an implementation-ready task list for the development intern. Implement, integrate, test and document the complete flow.

Product alignment: The supplied Snap Up PDD establishes customer self-checkout, barcode scanning, cart management, digital payment, digital receipts and secure exit verification. It also requires controlled backend-mediated access to retailer data and a 30-minute shopping session after Secure Dual Presence Authentication.

1. Database 2 → Database 1 Synchronisation

Objective: Use the supplied Database 2 API key to securely fetch required retailer product information and maintain a separate Snap Up-hosted Database 1 for customer-facing product retrieval.

Database 2 (Retailer Source) → Secure server-side connector → Sync Service → Database 1 (Snap Up)
→ Snap Up API → Customer Web App

Requirements

- Use the supplied Database 2 API key only on the server. Never expose it in frontend code, browser bundles, Git, logs or API responses.
- Fetch the required catalogue/product information from Database 2 through its authorised API.
- Database 1 must be a separate Snap Up-controlled customer-facing data store.
- Replicate the fields required for shopping: store, SKU/product, barcode, name, brand, weight/value, unit, price, availability and applicable discount/promotion information.
- Do not expose supplier information, procurement cost, margins, credentials or unnecessary retailer data.
- Synchronise Database 1 from Database 2 automatically once every 30 minutes.
- Make the sync idempotent so repeated runs do not create duplicates.
- Handle additions, updates, price/discount changes, availability changes and disabled/removed products.
- Record sync status, last success, duration, created/updated/disabled counts and failures.
- A failed sync must preserve the last known-good Database 1 state.
- Add indexes optimised for barcode/SKU/product retrieval.
- Customer APIs must never connect directly to Database 2.

Acceptance

A scanned product is resolved through the Snap Up backend and Database 1. Database 2 remains protected and the customer receives only the data needed to shop.

2. Automatic Session Renewal at T-60 Seconds

After successful 2-in-1 QR + authorised store-network verification, a 30-minute shopping session is created. The new requirement is to remove unnecessary interruption when the customer remains inside the store.

At approximately the final 1 minute, automatically recheck store-network presence. If valid, silently issue another 30-minute session without requiring another QR scan.

Requirements

- Trigger a background renewal/presence-check request at approximately T-60 seconds.
- The frontend timer is only a trigger; the backend is the authority.
- Verify session, store identity and authorised store-network presence server-side.

- If valid, issue one new 30-minute session without customer interaction.
- Do not display a disruptive modal when renewal succeeds.
- Update client expiry only after backend confirmation.
- If presence fails, do not extend the session.
- Prevent duplicate/parallel renewals from creating multiple sessions.
- Record renewal attempts and outcomes for audit/debugging.
- Client-side modification of time, expiry, store ID or payload must not bypass verification.

3. Guest Cart & Checkout Flow

Pilot rule: First-time customers can scan and shop as guests. Login is required when the customer wants to redeem the Snap Up discount.

Checkout requirements

- Validate the cart server-side before payment.
- Lock/freeze cart mutation while checkout validation is running.
- Never trust client-calculated totals.
- Calculate/validate item totals, service fees, discounts, GST/other charges and final amount on the backend.
- Preserve the cart securely during guest → login transition.
- Payment must use the backend-authoritative amount.

Guest checkout example

Payment Summary	Example
Item Total	■520
Service Fees	■40
Snap Up Discount	■0 — Login to redeem
GST & Other Charges	■17
Total to Pay	■577

Proceed to Pay → show the discount/login popup.

Popup behaviour

- Show the discount offer according to the UI/UX design supplied by the UI/UX team.
- Provide Login and Ignore actions.
- Ignore → continue payment at the current guest total.
- Login → redirect to login.
- After successful login, return to the same checkout page.
- The original cart and scanned products must remain unchanged.
- Do not create a new cart during authentication.

Logged-in checkout example

Payment Summary	Example
Item Total	■520
Service Fees	■40

Snap Up Discount	■40
GST & Other Charges	■17
Total to Pay	■537

- Recalculate the discount server-side after login.
- Validate the updated total before payment.
- Handle payment failure without losing the cart.
- Prevent duplicate payment attempts while a previous result is unknown.

4. Supplied Database + Barcode Catalogue Integration

Use the supplied Database 2 API key and the supplied Word barcode catalogue as the pilot data source. The customer web app must operate against the integrated data, not only hard-coded demo products.

- Synchronise/import the supplied catalogue into Database 1.
- Ensure every supplied barcode maps to the correct SKU/product variant.
- Test real barcode scanning through the customer web app.
- Verify product name, variant/weight, price, discount and availability.
- Test repeated scans and quantity increments.
- Test invalid barcodes, rapid scans and unavailable products.
- Test scanning with active, expired and lost-presence sessions.
- Verify Database 2 credentials never reach the browser.

5. Separate Customer Database & Pilot Authentication

Create a separate Snap Up-controlled database for customer/user information. It must be independent from retailer product data.

Pilot authentication: OTP is intentionally excluded. Use basic username/password authentication.

- First-time registration: Username, Password, Re-enter Password.
- Second-time login: Username, Password, Forgot Password.
- Passwords must be securely hashed; never store plaintext passwords.
- Apply basic rate limiting and secure session/token handling.
- Keep customer identity data separate from retailer product/catalogue data.
- Associate completed purchases with the authenticated account when logged in.
- Guest purchases must retain a secure shopping-session/transaction identity.
- Guest → login must attach the existing cart/session to the user instead of recreating it.

6. Payment → Exit QR → Staff Verification → Final Bill

After successful payment, the customer must **not** receive the final bill immediately. The customer first receives a secure exit/verification QR.

Required flow

Scan products → Cart → Checkout → Payment → Payment Confirmed → Exit QR → Staff scans QR → Staff sees cart + weight + amount → Deny / Proceed → Proceed releases final bill.

Staff portal requirements

- Provide a staff-side QR scanner page.

- The exit QR should contain a secure reference/token, not sensitive cart information in plaintext.
- Retrieve the corresponding transaction securely from the backend.
- Display purchased cart details required for verification.
- Display total weight and total paid amount.
- Display payment status and transaction reference.
- Provide Deny and Proceed actions.
- Deny must not release the final bill.
- Proceed authorises exit and final bill release.
- Staff actions must be authenticated, authorised, timestamped and auditable.

Final bill release

- Only after Staff → Proceed: mark the transaction exit-authorised.
- Generate/release the final digital bill with paid status/paid seal.
- Store the bill in bill history for the authenticated customer.
- Preserve immutable transaction references for customer, session, store, payment, receipt and exit authorisation.
- Prevent the same exit QR from being successfully processed twice.

7. Inventory Protection & Delayed Finalisation

Customer purchases must not cause product master/catalogue records to be deleted. Inventory is finalised only after staff verification.

Recommended state model

AVAILABLE → CART/RESERVED → PAID_PENDING_EXIT → EXIT_VERIFIED → FINALIZED

- Scanning adds an item to the cart without permanently deleting it from Database 1.
- Use reservation/pending state where necessary.
- After payment, retain transaction/product references while awaiting staff verification.
- Only Staff → Proceed triggers final inventory mutation.
- Staff Deny keeps the transaction in an auditable exception state.
- Never delete product master records because a customer purchased an item.
- Use server-side transactions/locking to prevent inventory race conditions.

8. Staff Store Wi-Fi Configuration

Add a staff/admin configuration section for each participating shop so Wi-Fi information can be maintained without changing source code or backend configuration manually.

- Provide Wi-Fi/SSID and password fields as required by the pilot UI.
- Save configuration against the specific store/branch.
- Use the latest active store configuration for presence verification.
- Never expose the stored Wi-Fi password through customer APIs.
- Do not hard-code Wi-Fi credentials in frontend source.
- Handle passwords securely and minimise unnecessary display after saving.
- Record configuration changes with actor and timestamp.

9. Full End-to-End Testing

Test the complete real application journey, not only isolated APIs.

Critical happy path

- Open customer web app and connect to participating store Wi-Fi.
- Scan the 2-in-1 QR and confirm a 30-minute session.
- Scan multiple real products from the supplied barcode catalogue.
- Verify correct product data and cart quantities.
- Test automatic renewal during the final 1 minute.
- Proceed to guest checkout and verify the payment summary.
- Test Ignore → payment at guest total.
- Test Login → original cart preserved → discount applied → updated total.
- Complete payment.
- Verify final bill is withheld and exit QR is displayed.
- Scan exit QR from staff portal.
- Verify staff sees cart, total weight and amount.
- Test Deny and confirm bill is not released.
- Test Proceed and confirm final paid bill is released.
- Verify bill history for logged-in customer.
- Verify inventory is finalised only after Proceed.

Security / negative tests

- Unauthorised product API access and expired-session access.
- Wrong store ID manipulation.
- Client-side price/discount/total manipulation.
- Exit QR replay and duplicate exit verification.
- Direct browser attempt against Database 2.
- Database 2 API-key leakage check in frontend bundle/logs.
- Wi-Fi disconnect or wrong store network before renewal.
- Payment timeout/unknown status.
- Cart mutation after checkout lock.
- Staff Proceed on an unpaid transaction.

10. Deliverables

- Database 2 → Database 1 secure synchronisation implementation.
- 30-minute scheduled sync with monitoring/status.
- Automatic final-minute session recheck and renewal.
- UI/UX-aligned guest and logged-in checkout.
- Guest → login → checkout cart preservation.
- Separate customer database and username/password authentication.
- Real supplied barcode/product integration.
- Backend-authoritative checkout and payment flow.
- Exit QR and staff verification workflow.
- Staff Deny/Proceed flow.

- Delayed inventory finalisation.
- Per-store Wi-Fi configuration.
- Unit/integration/E2E/security tests.
- Updated README and environment example files with no secrets.
- Final implementation report with completed work, test results, known limitations and assumptions.

11. Definition of Done

A feature is complete only when it works through the real application flow and is protected against obvious client-side manipulation. Hard-coded frontend data, exposed credentials, skipped backend validation, lost guest carts, premature bill release or premature inventory deletion are considered incomplete.

Final acceptance gate

- Build, lint and type checks pass.
- Critical automated tests pass.
- Real barcode/product data works through the customer web app.
- Database 2 credentials remain server-side only.
- 30-minute sync runs reliably.
- Final-minute presence renewal works without customer interruption when valid.
- Guest and logged-in checkout totals are correct.
- Guest cart survives login.
- Payment uses backend-authoritative amount.
- Final bill is withheld until staff Proceed.
- Inventory is finalised only after staff Proceed.
- Bill history is persisted for authenticated customers.
- Staff actions are authenticated and auditable.
- No secrets are committed to the repository.

12. Implementation Order

1. Database integration & sync → 2. Barcode retrieval → 3. Customer DB/authentication → 4. Guest cart & checkout → 5. Guest/login/discount transition → 6. Payment → 7. Exit QR & staff verification → 8. Inventory finalisation → 9. Session renewal → 10. Wi-Fi configuration → 11. Full E2E/security testing

Before coding: inspect the existing Snap Up repository and approved PDD, SERA, SEH, Technology Baseline, CLAUDE.md, RULES.md, PROJECT_STRUCTURE.md and CONTRIBUTING.md. Preserve existing architecture decisions unless a conflict is raised for review.

SNAP UP • Astra Dyne Global